

Optimizacija **LLVM** koda

Common Subexpression Elimination and Dead Basic Block Elimination with dominator analysis

Jovana Tasovac (27/2022) **Mina Protić** (80/2022)
Konstrukcija Kompilatora, Matematički fakultet u Beogradu

| Osnovni koncepti



LLVM IR

- Sta je LLVM?
- Faze LLVM-a
- SSA forma



CFG i Basic block

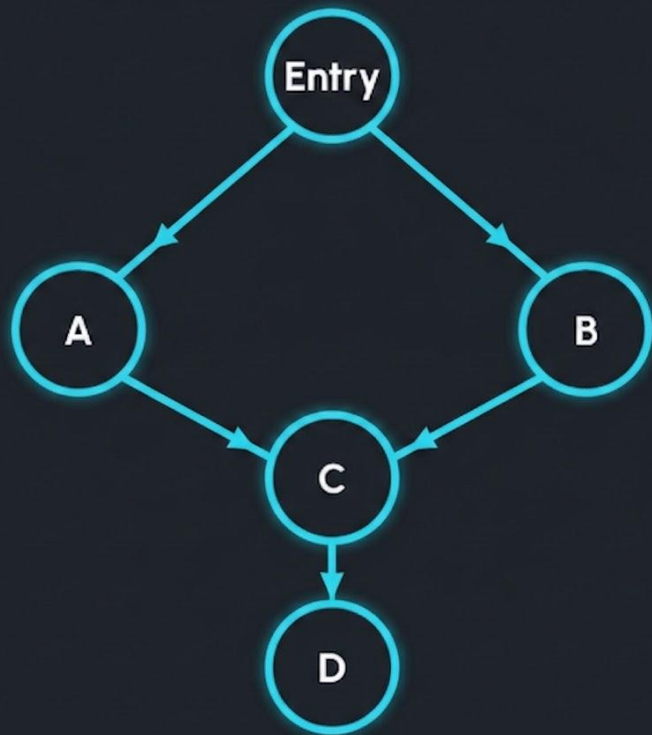
- (Control Flow Graph)
Graf kontrole toka ,cvorovi i grane
- Basic blocks



Optimizacije

- Middle-end
- Ne menjaju semantiku
- Poboljšavaju vremensku i prostornu složenost

| Analiza Dominacije



Stub obe optimizacije

- Blok A dominira blokom B ako svaki mogući put od ulaza do B mora proći kroz A.
- Znamo sa sigurnošću da se blok A izvršio pre bloka B.

| LLVM Prolazi i pokretanje

```
struct OurCSEPass : public FunctionPass {
    static char ID;
    OurCSEPass() : FunctionPass(ID) {};
    bool runOnFunction(Function &F) override {};
    //...
};

static RegisterPass<OurCSEPass>
X("cse-pass", "Our CSE Pass", false, false);
```

- tri vrste optimizacija: na celom modulu, na nivou funkcija i na nivou bloka
- obe naše implementacije su na nivou funkcije
- svaki function prolaz nasleđuje FunctionPass klasu i predefiniše funkciju runOnFunction()
- svaki prolaz mora da se registruje kako bi mu se mogli pristupiti prilikom poziva
- prilikom pokretanja .cpp kod se kompajlira do .so i zatim pokreće

```
opt -load ./lib/LLVM...so
-enable-new-pm=0 -nazivPass-a
<ulaz.ll> -S -o <izlaz.ll>
```

- kako bi se prolaz pokrenuo, prvo je potrebno da se kompajlira i dobije .so fajl
- učitavamo naš .so fajl sa -load
- -enable-new-pm=0 govori da koristimo stari pass manager
- -nazivPass-a je baš onaj iz RegisterPass()
- generišemo i tekstualni izlaz za naš primer

Common Subexpression Elimination based on dominator analysis

Cilj: ubrzanje izvršavanja programa kroz prepoznavanje i eliminaciju ponovljenih matematičkih izraza i suvišnih čitanja iz memorije

Algoritam:

- **Funkcije za analizu instrukcija:** `isInteresting`, `isFPType`
- **Optimizacija čitanja (load):** Ako je vrednost na adresi već učitana (i dominira), koristi se stari rezultat, a novi se markira za brisanje.
- **Invalidacija memorije (store):** Svaki novi upis na adresu automatski poništava njena prethodna učitavanja.
- **Optimizacija binarnih operacija:** `generateExpressionKey`, `isTheSameInstruction`, `isCommutative`; ako identičan izraz već postoji i dominira, nova instrukcija se preusmerava na originalni rezultat.
- **Faza čišćenja:** Bezbedno i konačno brisanje svih markiranih duplikata na kraju prolaza

`%1 = add i32 %a, %b`

⋮

~~`%2 = add i32 %a, %b`~~

replac

| Primeri izvršavanja

```
define i32 @test(i32 %a, i32 %b) {  
entry:  
  %x = add i32 %a, %b  
  %y = add i32 %b, %a  
  %z = add i32 %x, %y  
  ret i32 %z  
}
```

```
define i32 @test(i32 %a, i32 %b) {  
entry:  
  %x = add i32 %a, %b  
  %z = add i32 %x, %x  
  ret i32 %z  
}
```

CSE bez dominatora

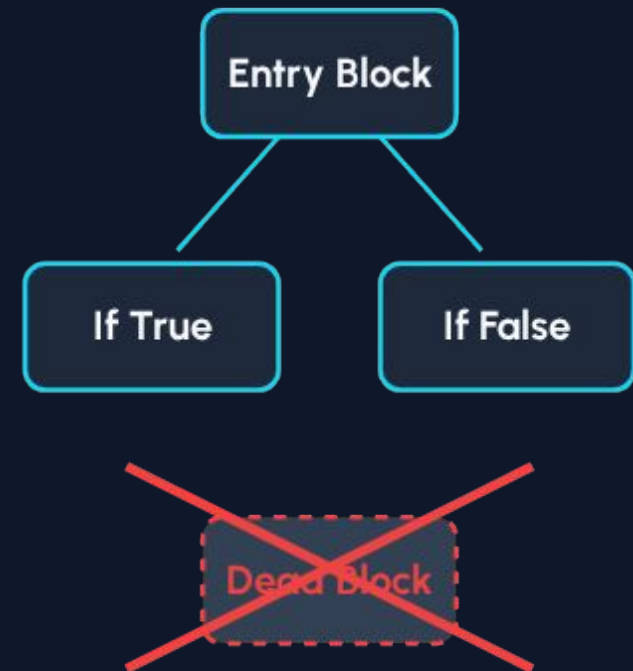
- bezbedan je samo na nivou bloka
- dominator omogućava bezbedno izvršavanje čak i kod skokova

Dead Basic Block Elimination based on dominator analysis

Cilj: smanjenje veličine koda i pojednostavljenje LLVM IR-a uklanjanjem mrtvih basic block-ova koji se nikada ne izvršavaju.

Algoritam:

- **DFS obilazak:** Obilazak kreće od entry bloka i pronalazi blokove do kojih postoji putanja.
- **Čuvanje dostižnih blokova:** Svi posećeni blokovi se pamte u unordered_set strukturi ReachableBlocks.
- **Analiza funkcije:** Pass prolazi kroz sve basic block-ove unutar jedne funkcije.
- **Provera:** Ako blok nije posećen DFS-om ili ga entry blok ne dominira, smesta se u vektor za uklanjanje
- **Uklanjanje mrtvih blokova:** prvo prolazim kroz sve sledbenike bloka, removePredecessor(BB) , dropAllReferences(), eraseFromParent()



| Primeri izvršavanja

```
define i32 @test_dead_block(i32 %x) {  
entry:  
  %cmp = icmp sgt i32 %x, 0  
  br i1 %cmp, label %positive, label %negative  
  
positive:  
  ret i32 1  
  
negative:  
  ret i32 0  
  
dead:  
  ret i32 100  
}
```

```
define i32 @test_dead_block(i32 %x) {  
entry:  
  %cmp = icmp sgt i32 %x, 0  
  br i1 %cmp, label %positive, label %negative  
  
positive:  
  ret i32 1  
  
negative:  
  ret i32  
}
```


Pitanja?

Hvala na pažnji i ukazanom vremenu.

Univerzitet u Beogradu, Matematički fakultet
Katedra za računarstvo i informatiku